Registers, Instructions, and the Instruction Cycle

Lecture Notes — Week 4

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

4.1 Introduction

In the first three weeks we built the *data* layer of a computer: how bits represent integers (two's complement) and reals (IEEE 754), and how adders and the ALU compute on those bits. But bits and adders do not run a program by themselves. Something must fetch an instruction, tell the ALU what to do, and move values between places. This chapter is about the **machine** that steps through an instruction: its storage spaces (registers, the stack), the format an instruction is laid out in, and the fetch–decode–execute cycle that animates them. It answers four questions that form one thread: what storage lives *inside* the CPU and how do the registers talk to each other and the ALU (Section 4.2); what it means to store operands in a LIFO stack and what hardware implements it (Section 4.3); how many addresses an instruction carries and what that choice costs (Section 4.4); and how the CPU steps a single instruction from fetch to execution (Section 4.5).

Throughout the chapter we compile the same expression, $X = (A + B) \times (C + D)$, in *every* instruction format we meet, counting instructions and memory accesses each time, then trace one instruction through the cycle it runs in. The four-format, four-count comparison at the end (Section 4.6) is the payoff: the format choice is a real, countable trade-off, not a matter of taste.

4.2 Register Organization

4.2.1 Motivation: Where Do Operands Live, Anyway?

A program keeps `X`, `A`, `B`, `C`, `D` somewhere. Before the ALU can add, values must move from where they are stored to the ALU's inputs — and back. How fast can each storage location respond? Main memory is big but slow: a read takes many CPU cycles. Registers are tiny but fastest — accessible in *one* cycle. Real CPUs keep the operands currently being worked on in registers, and the compiler decides what earns a register.

The CPU's major components are three (Figure 4.1): the **register set** holds the values, the **ALU** computes on them, and the **control unit** (built in Weeks 6–7) decides what happens when. Each part has exactly one job in the machine.

4.2.2 Program-Visible and Internal Registers

The registers a program *can* name are called program-visible (Mano, Sec. 8-2, p.242) [2]. This course uses three kinds. The **general-purpose** register file `R0`, `R1`, …, `Rn-1` holds operands and results; any register can hold an address or a value. The **accumulator** `AC` is the implicit operand/result

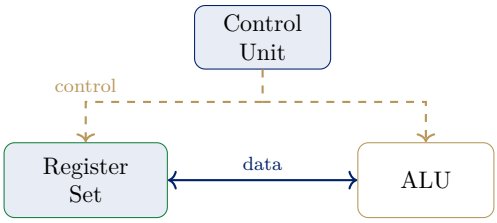

Figure 4.1: The CPU's major components — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 8-1, p.241: the register set, the ALU, and the control unit, with data and control paths [2].

register in one-address (accumulator) ISAs. The **stack pointer** `SP` holds the address of the stack top — the subject of Section 4.3.

The registers the program *never* names are the internal, invisible-to-programmer registers: `PC` (the address of the next instruction), `IR` (the current instruction), `MAR` (the address for the current memory access), and `MDR` (data in transit to/from memory). The control unit drives them through the instruction cycle; the program does not touch them directly. The same split appears in Patterson & Hennessy's LEGv8 treatment, where the architectural register set `X0`–`X30` plus `SP` is what a compiler names, while the fetch machinery lives in hardware the program never addresses (P&H, Sec. 2.3, p.67) [3].

Definition (Program-Visible Register). *A register that an instruction can name as an operand or a result location (e.g. `R0..Rn-1`, `AC`, `SP`), as opposed to the internal registers (`PC`, `IR`, `MAR`, `MDR`) that only the control unit reads and writes.*

4.2.3 How Registers Talk: the Common Bus System

Registers do not talk to each other directly; one shared path, the **common bus**, carries values among registers, ALU, and memory (Figure 4.2). A **select** unit (decoder) chooses, each cycle, *which* register drives the bus.

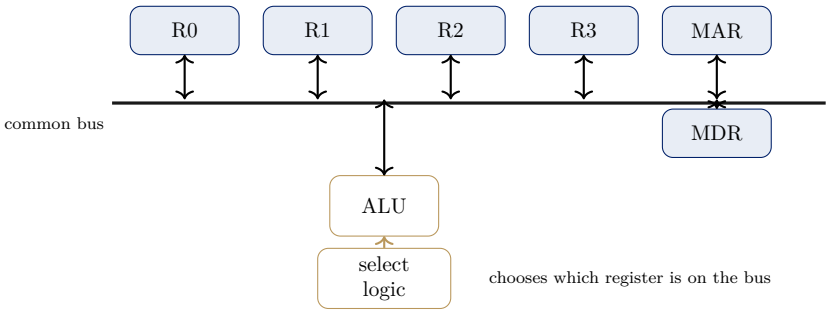

Figure 4.2: The common bus system — cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-4, p.129 (registers and memory on the common bus system). One shared path carries values among registers, ALU, and memory; a select unit (decoder) chooses which register drives the bus each cycle [2].

4.2.4 Register-Selection Signals: SELA / SELB / SELD

How does a transfer pick its source and destination? Mano's Table 8-1 (p.243) encodes the answer in three fields [2]. `SELA` names which register's contents go to the ALU's *first* input; `SELB` names

which register's contents go to the ALU's *second* input; and `SELD` names *which* register is loaded with the result.

The idea is that register selection is just *encoded* in a few select bits and decoded into one-hot "drive bus" / "load now" signals. That is the whole trick behind moving values between registers without hard-wiring every pair.

Example 4.2.1 (Copying `R1` ← `[R2]` over the common bus)**.** *The transfer request is to copy the* contents *of `R2` into `R1` (in the course's notation, the contents of a register X are written [X]). The bus-based mechanism is:*

1. *`SELA` encodes "`R2`" — R2's output enable goes high.*
2. *`R2`'s contents are driven onto the common bus.*
3. *`SELD` encodes "`R1`" — R1's load-enable goes high.*
4. *On the clock edge, `R1` captures whatever the bus holds:* $R1 \leftarrow [R2]$.

No direct `R2`→`R1` wire exists — every transfer uses the bus plus two select signals. The same mechanism drives every register move and ALU operand.

4.2.5 Socratic Check: Why Not Put *Everything* in Registers?

If registers are the fastest storage, why does a program put `X`, `A`, `B`, `C`, `D` in memory instead of keeping everything in `R0..Rn-1`? The answer has two parts. Registers are scarce: a few dozen at most — the program cannot keep megabytes of data there. And cost: each register costs silicon area and consumes power; large register files are expensive. So registers are a scratchpad for what is *being* worked on: values rest in memory (big, cheap, persistent) and are loaded into registers only while the ALU touches them.

4.3 Stack Organization

4.3.1 Motivation: How Do You Evaluate a Postfix Expression?

Humans write `A + B`; machines with one ALU must feed it two operands. How would a calculator evaluate `A B +` if it could only look at the *top* item of a scratch list — no registers, no random access? The rule: read a value, **push** it; read an operator, **pop** the last two values, compute, **push** the result. The *last* value pushed is the *first* one taken back out — **last-in, first-out** (LIFO). That is a **stack**. Many machines (HP calculators, the JVM) use exactly this model.

Definition (Stack). *A stack is an ordered storage structure where all insertions and deletions happen at* one end*, the **top**, addressed by a dedicated register `SP` (the **stack pointer**). Behavior is LIFO (Mano, Sec. 8-3, p.247) [2].*

Two operations manipulate the stack. **PUSH**: $SP \leftarrow [SP]-1$, then $M[SP] \leftarrow$ operand — store at the new top (the stack grows *downward* toward lower addresses). **POP**: operand $\leftarrow M[SP]$, then $SP \leftarrow [SP]+1$ — remove the current top.

Example 4.3.1 (PUSH and POP micro-transfers)**.** *Suppose the stack contains `[? , 5 , 7]` with top at 7. `PUSH 9` ⇒ stack `[? , 5 , 7 , 9]`; `POP` ⇒ returns 9, and the stack is back to `[? , 5 , 7]`.*

Example 4.3.2 (`A + B` on a stack machine)**.**

Step	*Operation*	*Stack after (top right)*	*Effect*
1	*`PUSH A`*	*`A`*	*value A stored at top*
2	*`PUSH B`*	*`B A`*	*value B stored above A*
3	*`ADD`*	*`A+B`*	*pop B, pop A, push result*
4	*`POP X`*	empty	*store top into `X`*

The `ADD` instruction names no operands at all — *both come implicitly from the top of the stack. This is the* **zero-address** *instruction format we meet in Section 4.4.*

4.3.2 Register Stack versus Memory Stack

There are two hardware homes for a stack (Mano, Sec. 8-3, p.248) [2]. A **register stack** lives inside the CPU in a set of registers — very fast, but only a handful of words (e.g. 64 words). A **memory stack** is a region of main memory, with `SP` holding the address of the current top — as large as memory, but every PUSH/POP is a memory access.

Real machines choose memory stacks because function calls nest deeply (recursion); a few registers cannot hold the frames. Real ISAs keep `SP` in a register and store stack frames in memory — registers are the pointer, memory is the stack. (P&H's LEGv8 makes exactly this choice: `SP` is a dedicated register and the procedure-call convention pushes frames onto memory, Sec. 2.8, p.100) [3].

4.3.3 Socratic Check: Registers versus Stack as the Operand Home

Both a register file and a stack can supply the ALU's operands. What does a stack force you to do that registers do not? Access is restricted: you can only reach the *top* — no random access to `R0..Rn-1`-style slots mid-computation. Reordering costs: to use a value that is buried, you must pop everything above it first (or shuffle). But instructions shrink: operands are implicit — hence the compact **zero-address** format. Compilers love the compactness; hardware pays with restricted access.

4.4 Instruction Formats

4.4.1 Motivation: What Must an Instruction Say?

The CPU must be told *what* to do and *where* the operands are. How much "where" information must a machine instruction physically carry? At minimum, an **opcode** — which operation (`ADD`, `LOAD`, …). Beyond that, *addresses* of operands and results — and each address costs bits in the instruction word. The trade-off is fundamental: **more addresses per instruction** ⇒ more work done per instruction but **longer instructions**; fewer addresses ⇒ compact but **more instructions** (and the hardware must know where the operands are implicitly).

Definition (Instruction Format). *An instruction format is the layout of an instruction word: an* ***opcode*** *field naming the operation, followed by zero or more* ***address/operand*** *fields naming where operands and results live (Mano, Sec. 8-4, p.255) [2].*

Fields are fixed-width bits; the opcode width sets an upper bound on how many distinct instructions the ISA can name. Address fields can name registers (short, cheap) or memory locations (long, flexible) — Week 5 turns those fields into real addresses via **addressing modes**. Patterson & Hennessy's LEGv8 shows the fixed-width field idea concretely: instructions are laid out in a

handful of formats (R/I/IS/B/CB/IM), each with fixed fields, and the opcode width bounds the instruction count (P&H, Sec. 2.5, p.82) [3].

4.4.2 The Four Format Families

As the operand count falls, instructions get shorter but the machine must find operands *implicitly* (accumulator or stack top). Figure 4.3 shows the four families as horizontal word layouts.

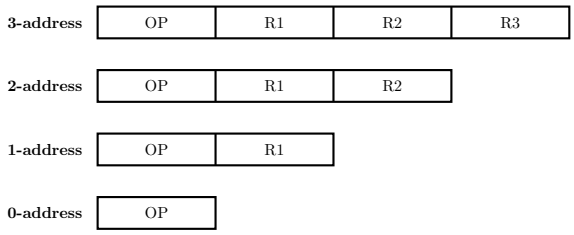

Figure 4.3: The four instruction-format families — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 8-4, p.255: as the operand count falls, instructions get shorter but the machine must find operands implicitly [2].

Three- and two-address forms. The 3-address form, `ADD R1, R2, R3` $\Rightarrow R1 \leftarrow [R2]+[R3]$, makes operands and destination all explicit — the most flexible, longest form. The 2-address form, `ADD R1, R2` $\Rightarrow R1 \leftarrow [R1]+[R2]$, lets the first field double as one operand *and* the result destination — this is **destructive**: the old value of `R1` is lost. Fewer fields $\Rightarrow$ shorter instructions and smaller programs, at the cost of destroying one operand and needing explicit copy instructions.

One- and zero-address forms. The 1-address (accumulator) form, `ADD A` $\Rightarrow AC \leftarrow [AC]+[A]$, always uses the accumulator `AC` as one operand (named by no field); the single field names the other operand (usually memory). The 0-address (stack) form, `ADD` $\Rightarrow$ pop two, push sum, carries *only* the opcode; every operand is implicit from the stack top. This is extreme compactness, but every non-stack access needs explicit `PUSH`/`POP` instructions.

4.4.3 Worked Examples: $X = (A + B) \times (C + D)$ in Every Format

The running thread makes the trade-off countable. In the **three-address** format the expression compiles to 3 instructions, operands all named:

Example 4.4.1 (3-address: `X = (A+B)×(C+D)`).

Instruction	*Effect*	*Notes*
`ADD R1, A, B`	$R1 \leftarrow A + B$	*operands and result all named*
`ADD R2, C, D`	$R2 \leftarrow C + D$	
`MUL X, R1, R2`	$X \leftarrow R1 \times R2$	

3 instructions; each carries three addresses — wide words, but the program is short and needs no extra data movement.

The **two-address** format needs 5 instructions because the destructive form requires copies:

Example 4.4.2 (2-address: the same expression).

Instruction	*Effect*	*Why*
`MOV R1, A`	$R1 \leftarrow A$	*explicit copy, since* `ADD` *overwrites*
`ADD R1, B`	$R1 \leftarrow R1 + B$	
`MOV R2, C`	$R2 \leftarrow C$	
`ADD R2, D`	$R2 \leftarrow R2 + D$	
`MUL X, R1`	$X \leftarrow R1 \times R2$	*R2 is the implicit second operand*

5 instructions vs. 3 — the 2-address format is cheaper per instruction but needs ***extra copy instructions*** *because* `ADD` *is destructive.*

The **one-address** (accumulator) format needs 7 instructions; a temporary `T` in memory is required because the accumulator cannot hold two partial results at once:

Example 4.4.3 (1-address: the same expression)**.** `LOAD A; ADD B; STORE T; LOAD C; ADD D; MUL T; STORE X` *— with a temporary* `T` *in memory because the accumulator cannot hold two partial results at once.*

The **zero-address** (stack) format needs 8 instructions, the most of any format, but each is the shortest possible:

Example 4.4.4 (0-address: the same expression)**.** `PUSH A; PUSH B; ADD; PUSH C; PUSH D; ADD; MUL; POP X` *— shortest instructions, but the most of them.*

4.4.4 The Four Formats Side by Side

Table 1 summarizes the four families. Same expression, four formats: 3, 5, 7, and 8 instructions. More addresses per instruction ⇒ fewer instructions but wider words; fewer addresses ⇒ more instructions but compact.

Format	**Example**	**Program size for $X = (A+B) \times (C+D)$**	**Operand access**	**Typical use**
3-address	`ADD R1,R2,R3`	3 instr., wide words	all explicit	register ISAs (RISC)
2-address	`ADD R1,R2`	5 instr. + copies	one destroyed	older CISC
1-address	`ADD A`	7 instr., needs temps	accumulator	simple CPUs
0-address	`ADD`	8 instr., compact	stack top	stack machines, JVM

Table 1: Instruction formats side by side — cf. Mano, *Computer System Architecture*, 3rd ed., Sec. 8-4, p.255 [2].

4.5 Instruction Cycle

4.5.1 Motivation: Who Runs the Show, Step by Step?

`ADD R1, A` sits in memory. For it to take effect, the CPU must bring the instruction in, understand it, collect `[R1]` and `[A]`, add, and store back. The answer is a fixed skeleton — the **instruction cycle** — that the control unit runs over and over, once per instruction. The skeleton is the *same* for every instruction; only the **execute** phase differs by opcode.

Definition (Instruction Cycle). *The fixed sequence of phases — fetch, decode, execute — the control unit repeats once per instruction; only the execute phase varies with the opcode.*

4.5.2 The Cycle: Fetch, Decode, Execute

Fetch → decode → execute is the standard instruction-cycle skeleton (Hamacher, *Computer Organization*, Ch. 7; standard treatment) [1]. Figure 4.4 shows it as a loop.

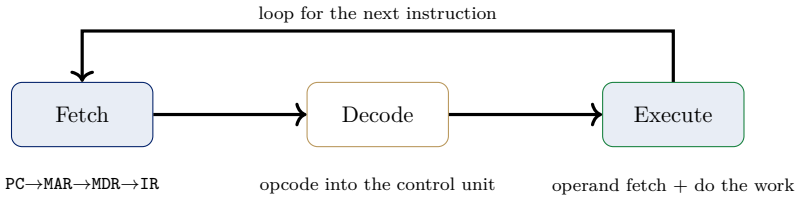

Figure 4.4: The instruction cycle — fetch, decode, execute — as a loop (Hamacher, *Computer Organization*, Ch. 7; standard treatment) [1].

4.5.3 The Fetch Phase, Step by Step

The fetch phase moves the next instruction from memory into the CPU (`PC` knows where the next instruction lives):

- `MAR` ← `[PC]` — remember the address.
- Initiate a memory *read* at `MAR`.
- `MDR` ← *memory data* — the instruction word comes back into the CPU.
- `IR` ← `[MDR]` — the instruction now sits in the instruction register.
- `PC` ← `[PC]` +1 — prepare the *next* fetch.

Why these registers? `PC` holds the next address, `MAR` pins it during the access, `MDR` buffers the incoming word, `IR` holds the current instruction while it is decoded. Each register exists for *one* reason in the cycle.

4.5.4 The Decode and Execute Phases

Decode. The opcode bits of `IR` tell the control unit *which* operation this is — the unit then selects the right sequence of control signals (exactly what we build in Weeks 6–7).

Execute is the operand-dependent part. If a memory operand is needed, its address is resolved (addressing modes, Week 5) and the operand fetched into a register. The ALU performs the operation; the result is written to the destination named in the instruction. The *number* of execute steps varies by instruction — fetch is fixed, execute is not.

Example 4.5.1 (One full cycle for `ADD R1, A`).

Phase	***Register activity***	***Memory / ALU***	***State***
Fetch	*MAR←[PC]; MDR←instr.; IR←[MDR]; PC←[PC]+1*	*read instruction word*	*instruction in IR*
Decode	*IR opcode → control unit*	—	*operation identified*
Execute	*operand [A] fetched; AC←[AC]+[A]*	*read operand; ALU adds*	*result in AC*

The same three phases run for every instruction — only the execute row changes. This is the loop the CPU is always inside.

4.5.5 Types of Operands

What the fields of an instruction can name (Hamacher, Ch. 7; standard treatment) [1]:

- **Addresses**: a location in memory (used by `LOAD`, `STORE`, `BRANCH`).
- **Numbers**: integer or floating-point values the ALU computes on.
- **Characters**: text — usually ASCII/Unicode codes stored in a byte or halfword.
- **Logical data**: bit patterns — each bit is its own value; operations like `AND`, `OR`, `XOR`, `shift` work bitwise.

Why this matters: the *type* determines how the ALU wires the bits. A branch needs the address, an integer add needs the value, a shift needs the bit mask. The opcode says which interpretation applies. (P&H's LEGv8 makes the same point in its operand taxonomy — registers, memory, constants — Sec. 2.3, p.67, and the branch/decision instructions of Sec. 2.7, p.93) [3].

4.5.6 Socratic Check: Tying the Week Together

For `ADD R1, A` on an accumulator machine: which register holds the next instruction's address, which holds the current instruction, and where do the two operands of the ADD come from? `PC` points at the next instruction; `IR` holds the current one after fetch. The first operand is `AC` (the accumulator — the implicit operand of the 1-address format). The second operand is memory location `A`, fetched during execute. Every piece of this answer came from the preceding sections — the registers, the formats, and the cycle are one connected machine.

4.6 Synthesis and Summary

4.6.1 The Three Threads Meet

Storage spaces and formats are two sides of one choice. **Registers**: fast, explicit, random access — buys the wide 3-address format. **Stack**: compact, restricted, implicit — buys the 0-address format. The instruction cycle is the *glue*: it steps whichever format the ISA chose, through the same fetch/decode/execute skeleton.

The running thread closes: $X = (A + B) \times (C + D)$ ran in 3, 5, 7, or 8 instructions depending on the format — and each of those instructions then lives through one full fetch–decode–execute cycle.

4.6.2 Bridge to Week 5

What we built this week: the CPU's storage (registers — fast scratchpad; stack — LIFO operand home; memory — big and slow); four instruction formats with a counted cost per format; and the fetch–decode–execute cycle with the four types of operands. What is still missing: we wrote "`ADD R1, A`" and said "operand `[A]` is fetched" — but we never said **how** the field `A` turns into the actual location of the operand. That is **addressing modes**, next week — together with the datapath (buses) that actually moves the values.

4.6.3 Summary

Register organization: registers are the one-cycle scratchpad; transfers happen over a common bus selected by `SELA`/`SELB`/`SELD`. **Stack organization:** `SP` addresses a LIFO top; PUSH/POP with ± 1 on `SP`; register stacks are fast, memory stacks are large. **Instruction formats:** 3-, 2-, 1-, 0-address — the address count trades instruction width against program size (3 vs. 5 vs. 7 vs. 8 instructions for one expression). **Instruction cycle:** fetch (`PC`→`MAR`→`MDR`→`IR`, `PC`+1), decode, execute. **Operand types:** addresses, numbers, characters, logical data — the opcode picks the interpretation.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.

[3] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.